

ДОБАВИЛИ МОДУЛИ ПО РАБОТЕ С НЕЙРОСЕТЯМИ В ЧАСТЬ ПРОГРАММ

Теперь и на курсе «Мидл системный аналитик»: учим выбирать, промптировать и проверять ИИ

[НАЧАТЬ БЕСПЛАТНО](#)

Я Практикум PRO

**128K+**

Охват за 30 дней

Яндекс Практикум

Помогаем стартовать и расти

144 Рейтинг

125 682 Подписчики

[Подписаться](#)**SddForever**

1 час назад

Искусственный интеллект в Data Science: инструменты и границы возможностей

😊 Простой 🕒 8 мин 👁 1.4K

Блог компании Яндекс Практикум, Учебный процесс в IT, Искусственный интеллект, Data Engineering*, Анализ и проектир

[Мнение](#)

Нейросети ещё не стали обязательным пунктом в требованиях работодателей, но на практике многие дата-сайентисты уже используют LLM в работе: генерируют код в Cursor, пишут SQL-запросы в DataGrip и спрашивают совета у ChatGPT.

Хабр, привет! Меня зовут Вячеслав Демин, я больше пяти лет работаю в сфере Data Science. Сейчас я руководитель направления аналитики данных в Сбере и эксперт на курсе [«Специалист по Data Science»](#) в Яндекс Практикуме. Начинал с этого же курса в 2020 году, после чего работал в сфере страхования и нефтехимии.

В этом материале я расскажу, для каких задач в Data Science использую ИИ и почему полагаться на него на 100% всё ещё не стоит.

Для каких задач в Data Science используют нейросети

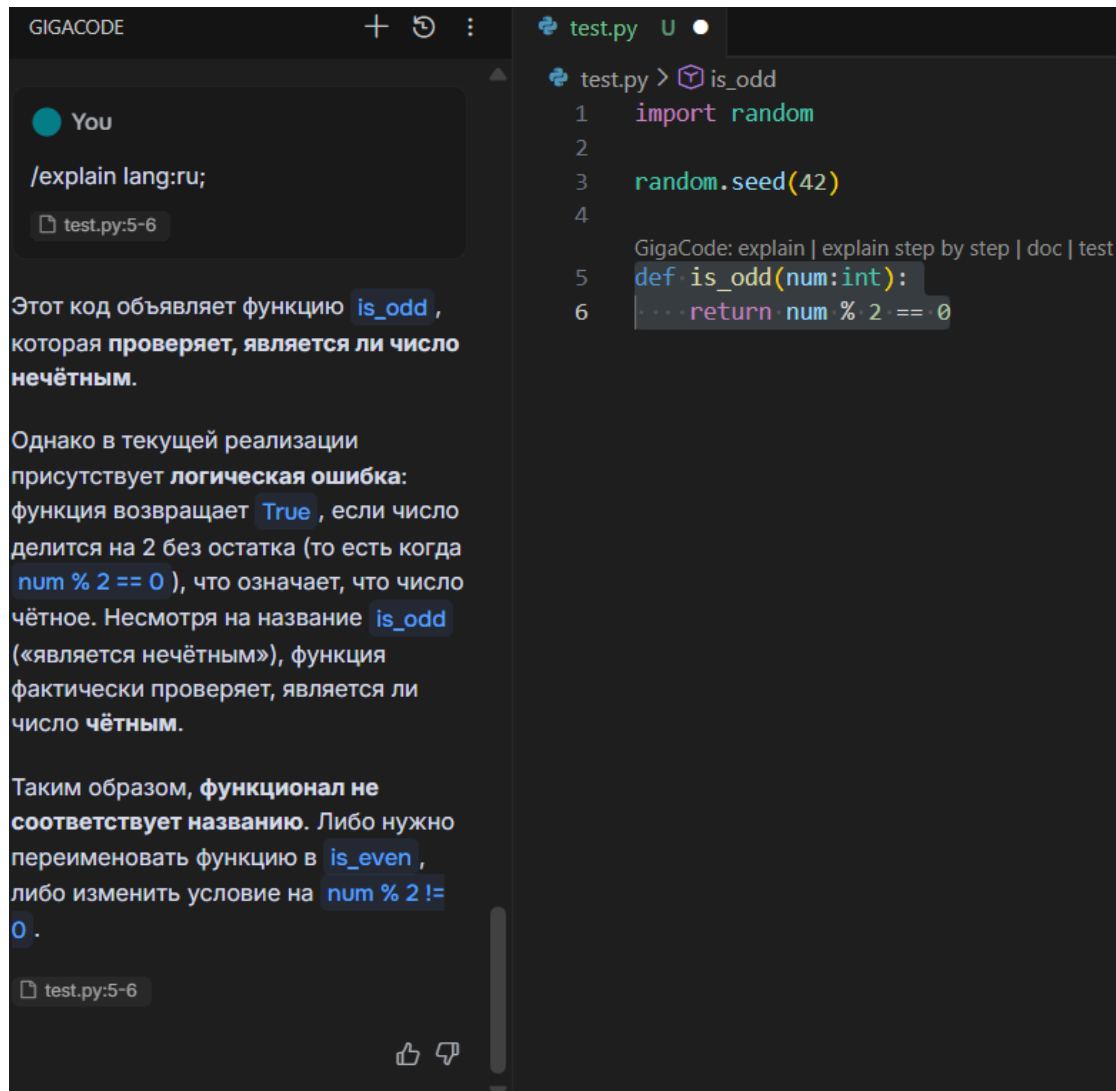
Генерация и сопровождение кода

Самое популярное направление применения ИИ в разработке — работа с кодом: автодополнение, генерация фрагментов, объяснение логики, рефакторинг и генерация тестов. Нейросеть легко объяснит работу непонятной секции кода, переведёт фрагмент с одного языка на другой, продолжит начатую строку или построит за разработчика docstring — описание функции, класса или элемента.

```
test.py U ●
test.py
1 import random
2
3 random.seed(123)
```

Здесь GigaCode пытается предложить число в параметр `seed()`

Мой главный инструмент для этих задач — GigaCode, ассистент для написания кода на базе GigaChat. По пользовательскому сценарию он близок к GitHub Copilot: помогает дописывать код, объяснять фрагменты и ускорять рутинные действия. Разница в том, что Copilot обычно глубже встраивается в привычный IDE-стек через расширения, а GigaCode требует отдельного подключения и настройки.



По добавленному контексту над функцией нажал explain, и в окне слева отправился запрос в GigaChat с объяснением

Есть приложения, такие как Cursor, в которых можно настраивать промпты для написания кода. Это полноценная среда, где можно задавать инструкции и контекст для генерации кода (например, описывать стиль, требования или задачу). Такие инструменты работают с контекстом проекта — анализируют открытые файлы и релевантные части кода. Теоретически можно даже дать им задачу на рефакторинг всего проекта, хотя на практике такой запрос потребует поэтапного контроля и ревью.

Пример работы в Cursor

Пример работы в Cursor

В эту же категорию можно отнести работу с SQL-ассистентами, которые помогают сгенерировать и оптимизировать запросы, подсказать, как собрать их в удобном и читаемом виде. Тут особенно важно следить за работой ИИ — дата-сайентист должен понимать, по какой логике собраны столбцы, поля и строки, какая фильтрация используется. Нейросеть часто что-то пропускает.

Интерфейс DataGrip

Интерфейс DataGrip

Для SQL-задач можно использовать как универсальные ИИ-ассистенты, так и специализированные инструменты. Например, DataGrip удобен для работы с запросами, рефакторинга и анализа плана выполнения, а платформы вроде Microsoft Fabric дополняют работу ИИ-функциями на уровне аналитической среды. Мне особенно нравится DataGrip с его возможностями рефакторинга запросов, то есть оптимизации уже написанного кода. А ещё он позволяет посмотреть execution plan и понять, как СУБД будет выполнять запрос, без дополнительных запросов и ручной проверки.

Написание документации

ИИ отлично проявляет себя в работе с текстами. Например, генерирует базовые файлы для проекта — README и docstrings для всех функций.

Предложенный ReadMe.md

Заполнение docstrings после описания всех параметров функции

Также большинство современных моделей могут помочь с саммари переписок и документов с требованиями от заказчика — так, чтобы их удобно было приложить к тикету в CRM.

Создание тест-кейсов

ИИ хорошо дополняет разработчика при создании тест-кейсов — особенно в генерации крайних случаев и неочевидных сценариев. Разработчику трудно предсказать все ситуации, в которых с кодом может пойти что-то не так. Неважно, идёт ли речь о разработке, создании модели или пайплайна — ИИ может задать больше условий и обнаружить скрытые уязвимости и ошибки.

Написание простых сервисов

Современные LLM, такие как ChatGPT, Claude, Алиса AI, GigaChat, Qwen Chat или DeepSeek, подходят для быстрых решений небольших задач, например прототипирования простых сервисов. Особенно полезно, если знания дата-сайентиста в этой области теоретические и ему недостаёт практики.

Использование агентов

Отдельный класс решений — агентные пайплайны, в которых LLM не просто отвечает на запрос, а выбирает следующий шаг, вызывает инструменты и передаёт результат дальше по цепочке. Чаще

всего их создают с нуля с помощью LangChain и LangGraph — получаются готовые ИИ-ассистенты под любую задачу.

Например, граф какой-либо оркестрации. Ты задаёшь агенту вопрос, а он определяет тип задачи и направляет тебя в конкретную секцию, граф или ноду с промптом, который будет работать с этой темой. Чаще всего агентов так и используют, для маршрутизации — то есть чтобы выстроить цепочку с автоматическим выбором действий после обработки запроса.

Почему нейросетям нельзя доверять на 100%

Есть несколько причин, почему нейросети ещё не заменили и вряд ли заменят живых дата-сайентистов — у всех ИИ-инструментов есть несколько ограничений.

Не учитывают весь контекст

ИИ хорошо проявляет себя в локальных задачах: описать функцию, оптимизировать фрагмент кода, предложить продолжение конкретной строки. Но он не может «держать в уме» всю модель системы, которую видит разработчик: не знает все требования от заказчика и нефункциональные ограничения, которые ИИ учитывает значительно хуже.

Предлагают нестандартные решения (не всегда лучшие)

ИИ не решает задачи с нуля — он опирается на паттерны из обучающих данных. Иногда это хорошо, иногда не очень. В коде могут быть и скрытые баги, и просто нестандартные решения, которые будет трудно объяснить команде или дорабатывать после релиза.

Пример из моего опыта: однажды в фрагменте кода на Python Copilot предложил мне результат с условием `else` на выход из цикла. Такая конструкция технически корректна, но используется редко и часто плохо читается, поэтому в продакшен-коде её обычно избегают.

Использование `for-else` в Python

Как лучше писать такие условия выхода без `for-else`

Разработчику нужно быть готовым, что модель предложит работающее, но нестандартное решение. Обычно его лучше заменить на более привычную альтернативу, ну или хотя бы попытаться выяснить, почему нейросеть решила написать код именно так.

Выдают ошибки

Частая проблема: код вроде бы работает, но логически неверен.

При генерации на Python ИИ может неправильно проставить условие в строчку `if`, добавить что-то лишнее, некорректно считать список свойств из другого сервиса через API или вызвать исключение, которого не было в инструкции.

Некорректное указание LLM — и промпт генерирует признак с target leakage

Результаты получения target leakage, где все значения в тесте пустые или неадекватные

Другой вариант: когда ИИ фокусируется на конкретной части запроса, игнорируя прочие условия задачи. Допустим, твоя метрика — 0,7, а нужно, чтобы была 0,95. Нейросеть выдаёт все 0,98, но на деле ломает код: в одном из признаков дублирует таргет (target leakage), из-за чего всё работает на тестах, но на раз-два заваливается на проде.

Вольно обращаются с источниками

Если мы говорим о Data Science, нейросети допускают ещё одну непростительную для нас ошибку — могут ссылаться на недостоверные или нерепрезентативные источники, используя данные без подтверждения статистической или научной обоснованности. Например, в ответ на общий запрос «расскажи мне об исследованиях в этой области» модель может выдать тебе за общие выводы информацию из маленького абзаца в материале. Причём материалом может быть даже не исследование, а просто статья в интернете.

Копят недочёты

Условный Copilot может сгенерировать функцию, которая будет работать, но код при этом будет написан не оптимально. Разработчик может не заметить недочёт и оставить его в коде. Со временем такие решения начинают тиражироваться внутри команды или проекта — и становятся локальным «стандартом», даже если они не оптимальны. Как стажёр, которого учат исключительно на плохих примерах.

Как работать с нейросетями без потери качества результата

Чтобы избежать типовых ловушек в работе с ИИ, важно соблюдать несколько полезных привычек.

Декомпозировать задачи

Чем проще запрос, тем надёжнее срабатывает ИИ. То есть добавить к коду пару строчек для него легко, а вот выполнить многосоставную задачу сложнее — такой подход может привести не только к долгим рассуждениям, но и к постепенному накоплению ошибок в процессе и некорректному результату.

Челленджить ИИ в работе с данными

Проверяя статистику и выводы ИИ, не стоит принимать его ответы на веру. Лучше обратить его внимание на неочевидные моменты и спросить: почему ты так думаешь? Зачем это нужно? А откуда ты это взял?

До прихода ИИ дата-сайентисты всегда использовали математические подходы и статистические оценки, чтобы оценить правдоподобность результатов экспериментов. От них не стоит отказываться.

Проверять результаты работы одной LLM с помощью другой

При работе с кодом логично использовать разные модели. LLM может быть менее критична к решениям, построенным по знакомым ей паттернам. Чтобы избавиться от эффекта стажёра («я постарался, и теперь код работает, а значит, всё хорошо»), логично обращаться к другой модели, чтобы она оценила работу более критично.

Валидировать результат вручную

Когда занимаешься критичными процессами, важно проверять работу LLM не только силами другой LLM, но и с участием живого разработчика. Это справедливо в отношении всех задач, которые касаются безопасности, криптографии и прав доступа — здесь любая фантазия может привести к неправильной авторизации. Или, к примеру, при работе с генерацией запросов на давно спроектированных базовых витринах. Вложенность в таких структурах может быть бесконечной — ошибка в одной приведёт к тому, что финальная витрина просто не соберётся. ИИ в таких задачах может выступать только как подсказчик.

Кроме того, разработчик должен в любом случае понимать значение каждой строчки сгенерированного кода. Допустим, результат выглядит и работает корректно, но что если в нём

используются специфические библиотеки или функции, которые не заработают на другой машине? А если он просто не оптимален?

Даже если ты настраиваешь свою LLM под свой тип задач и в двух случаях из трёх она не галлюцинирует, это не гарантирует, что она не ошибётся в третий раз — просто потому что нейросеть сработала иначе. Так что валидация работы ИИ живым разработчиком обязательна, по крайней мере пока.

Проверять воспроизводимость результата в чистом окружении

Любой код нужно проверять на нескольких устройствах. Обычно мы отправляем проект через Git — в GitHub, GitLab, Bitbucket и другие сервисы. Если в проекте нет requirements, тестов или другой конфигурации, которая позволяет воспроизвести окружение на другом устройстве, это почти гарантированно приведёт к проблемам. В таком случае код может не запуститься на соседнем ноутбуке.

Особенно это критично при работе с ИИ: без чётких инструкций и настроек он может сгенерировать решение, которое невозможно корректно воспроизвести в другой среде.

Задавать ограничения

Наверное, самое важное: в 2026 году ключевой навык — это не просто умение писать хорошие промпты. Да, важно задать роль, прописать инструкцию, ограничения и привести примеры. Но ещё важнее — уметь проверять и ограничивать результат.

Например, в LangChain есть удобный формализованный подход. Ты задаёшь системный промпт и можешь дополнить его «мини-промптом», который повышает вероятность корректного ответа.

Вариант реализации с парсером

Как это обычно устроено:

- создаётся промпт и параметры,
- всё это отправляется в LLM,

- на выходе результат обрабатывает парсер.

Парсер задаёт чёткий формат ответа. Ты заранее указываешь, какие именно данные ожидаешь — их может быть больше или меньше, но структура всегда фиксирована. За счёт этого результат становится гораздо стабильнее.

Альтернативный способ получить предсказуемый ответ — использовать встроенный структурный вывод в LangChain через `with_structured_output()`. В этом подходе мы не просто просим модель «ответить в JSON», а заранее описываем ожидаемую схему результата — например, через Pydantic-модель или JSON Schema. После этого LangChain оборачивает вызов модели так, чтобы на выходе мы получали не произвольный текст, а объект с заранее известными полями и типами. Это снижает количество ошибок парсинга, упрощает последующую обработку результата в коде и делает поведение LLM стабильнее в продакшен-сценариях.

Почему джунам лучше выполнять задачи самостоятельно

Я пока не видел ни одной вакансии, где знание и использование ИИ было бы обязательным требованием. Это точно полезный навык, но попасть в Data Science можно и без него.

А ещё нейросети могут усложнить решение новых и пока ещё сложных задач для начинающих дата-сайентистов. Допустим, у нас есть два джуна, которые решают одну задачу. Первый работает по классической схеме, а второй использует ИИ.

Первый джун исследует документацию, собирает код по подсказкам со Stack Overflow и обращается к мидлу из своей команды, когда возникают проблемы. В итоге через неделю приходит с кодом, который работает корректно и даже частично валидирован старшим коллегой.

Второй выдаёт результат — готовую задачу уже через два дня. В тестах оказывается, что код работает не по ТЗ, а джун плохо ориентируется в том, что написал. Задачу приходится переделывать — в итоге она занимает даже больше времени, чем у дата-сайентиста без ИИ-помощников.

Это утрированный пример, но суть передаёт верно: ИИ ускоряет одни процессы, но может замедлить другие и увеличить путь до финального результата. Поэтому пользоваться им можно и нужно, но только с пониманием, как именно работает генерация, и с готовностью проверить и доработать результат вручную.

Резюмирую: Copilot и другие инструменты ускоряют работу, но не заменяют мышление. ИИ — это помощник, которому нужно поручать простые задачи, внимательно направлять и проверять результаты. И помнить: ответственность за результат работы LLM всегда остаётся на человеке.

P.S. А теперь скажите, где тут настоящий скриншот :-)

**Практикум**

Учитесь в условиях
настоящей работы

Обучение на курсе «Специалист по Data Science» разделено на спринты: у вас будут свои проекты, ревью, мягкие и жёсткие дедлайны.

Попробовать




Теги: аналитика, cursor, copilot, data science, datagrip, gigacode, нейросети для разработчиков, ии-агенты, ии-агенты для разработки, ии в разработке

Хэбы: Блог компании Яндекс Практикум, Учебный процесс в IT, Искусственный интеллект, Data Engineering, Анализ и проектирование систем



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



128K+

Охват за 30 дней

Яндекс Практикум

Помогаем стартовать и расти

[Сайт](#) [Telegram](#)



2

Карма

Демин Вячеслав @SddForever

DS 5+ года опыта

Подписаться



Хабр Карьера

Комментарии 1

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники



Простой



4 мин



16K



+96



30



27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно



Простой



11 мин



26K

Обзор

Перевод



+68



45



81



prohronus

18 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере



5 мин



9.2K

Кейс



+51



9



9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц



Простой



7 мин



12K

Обзор



+37



59



18

 **beget_com**
23 часа назад

Механический калькулятор. Как работает арифмометр?

 Средний  10 мин  9.5K

Обзор

 +37

 23

 8

 **TrexSelectel**
19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин  8.7K

Обзор

 +27

 10

 0

 **AlexSerbul**
23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний  14 мин  9.9K

Мнение

 +26

 33

 11

 **mariklolik**
18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

 Средний  7 мин  7.4K

Кейс

 +23

 23

 6

 **nick_dyuba**
22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и ТіріТір

 5 мин  7K

Recovery Mode

 +22

 8

 5

 **Mimizavr**
23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

 11 мин  6.9K

 +17

 9

 7

Показать еще

ИНФОРМАЦИЯ

Сайт	practicum.yandex.ru
Дата регистрации	9 ноября 2020
Дата основания	12 февраля 2019
Численность	101–200 человек
Местоположение	Россия
Представитель	Ира Ко

ССЫЛКИ

[Бесплатный курс «Основы математики»](#)
practicum.yandex.ru

[Бесплатный курс «Наставник в IT»](#)
start.practicum.yandex

[Бесплатный курс «Подготовка к алгоритмическому собеседованию»](#)
start.practicum.yandex

[Бесплатный курс «Основы Go»](#)
start.practicum.yandex

[Бесплатный курс «Основы Python-разработки»](#)
start.practicum.yandex

[Бесплатный курс «Основы работы с базами данных и SQL»](#)
start.practicum.yandex

[Бесплатный курс «1С: программирование на русском»](#)
start.practicum.yandex

ВИДЖЕТ

 Практикум

Курсы
от авторов-
практиков

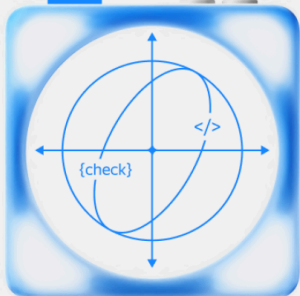


ВИДЖЕТ

 Практикум

Курсы
с проектами
и код-ревью

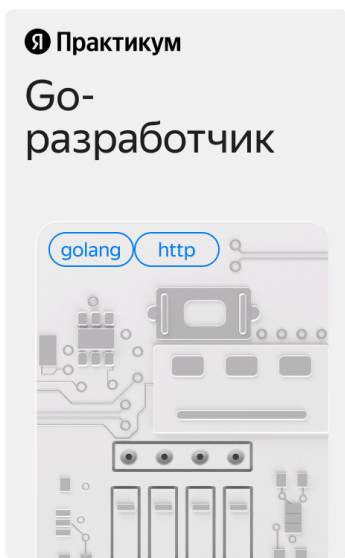
РАСТЕМ



ВИДЖЕТ



ВИДЖЕТ



БЛОГ НА ХАБРЕ

1 час назад

Искусственный интеллект в Data Science: инструменты и границы возможностей

1.4K 1

15 апр в 14:15

«Приходила уставшая, ставила таймер на два часа и училась»: как я стала DevOps-инженером

16K 13

14 апр в 11:02

Как дела в коммерческом геймдеве в 2026 году

11 апр в 11:55

Продакт в 2026 году: чем занимается, как им стать и почему цифровому бизнесу без него никуда

10 апр в 11:02

Почему сильный разработчик не всегда становится сильным тимлидом — и что с этим делать

Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам









Настройка языка

Техническая поддержка

© 2006–2026, Habr